

PRODUCT REQUIREMENTS DOCUMENT

****Project Title:** Personal Financial Planning System**

****Course:** WIF2003 Web Programming (Semester 2, Session 2025/2026)**

****Team Name:** Team 04 (Occ 1)**

****Team Leader:** Tan Yin Xuan**

Team Members & Module Assignments

| Team Member | Role | Primary Module(s) |

|---|---|---|

| Tan Yin Xuan | Team Leader | Module 2: Cash Flow & Budget Baseline; Module 5: Market Insights & Analysis |

| Ajax Kang AJ | Member | Module 1: Authentication & Core Backend Infrastructure |

| Lai Onn Wang | Member | Module 1: Authentication & Core Backend Infrastructure |

| Lee Yi Chen | Member | Module 3: Goal-Based Planning |

| Cheng Tze Xuan | Member | Module 4: Investment Strategy |

| Veronica Chieng Xin Ru | Member | Module 6: Financial Calculators |

> ****Note:**** Module 1 is jointly owned by Ajax Kang AJ and Lai Onn Wang. Responsibility split: Ajax owns the Express server scaffolding, JWT middleware, and auth routes; Lai owns Mongoose connection, global error handling, and protected route middleware.

1. Executive Summary

The Personal Financial Planning System is a standalone, educational web application engineered to empower individual users to track their financial health, set concrete funding goals, and explore fundamental investment strategies. It relies entirely on user-inputted data rather than automated bank synchronisation. The system prioritises a highly professional UI/UX, robust data security, and seamless integration with external market data APIs to deliver dynamic financial insights.

1.1 Target Audience & Constraints

****Target Audience:**** Individual users looking to organise personal finances and understand basic market trends.

****Out of Scope:**** The system explicitly does not support enterprise-level financial management, does not integrate with real banking systems, and does not provide professional or licensed financial advisory services.

****Assumptions:**** Users have basic financial literacy, are responsible for accurate manual data entry, and understand that strategy recommendations are based on predefined logic, not guaranteed outcomes.

2. System Architecture & Tech Stack

To demonstrate mastery of web programming skills and secure maximum marks, the system uses a modern, decoupled MERN architecture, strictly adhering to the permitted frameworks.

2.1 Frontend (Client-Side)

React (powered by Vite) utilising HTML5, CSS, and modern ES6+ JavaScript/JSX.

2.2 State Management & Form Handling

****Zustand:**** Used strictly for lightweight global state management (e.g., UI themes, authenticated user session object, and auth status flag). Zustand is the single source of truth for client-only UI state.

****TanStack Query:**** Handles all asynchronous server state exclusively — caching and background fetching for external market APIs, and any data originating from the backend (e.g., Goals, Cash Flow). Query configuration:

- Market data queries (Module 5): `staleTime: 5 \* 60 \* 1000` (5 minutes), `gcTime: 10 \* 60 \* 1000`

- User data queries (Goals, CashFlow, Profile): `staleTime: 60 \* 1000` (1 minute), `gcTime: 5 \* 60 \* 1000`

****react-hook-form + Zod:**** Manages all form state and client-side validation across every module to maintain consistency and prevent malformed API calls.

> ****State Boundary Rule:**** If data comes from the server or an API, it belongs in TanStack Query. If it is local UI state, it belongs in Zustand. Modules must consume shared server data via custom TanStack hooks (e.g., `useCashFlow()`) to prevent state duplication.

2.3 Styling Strategy

Raw Bootstrap SCSS will be utilised. To achieve the custom themes, the team will compile custom SCSS variables to override default Bootstrap variables before compilation. The final report must include the relevant `vite.config.js` configuration and SCSS entry file as evidence.

2.4 Backend (Server-Side)

****Framework:**** Node.js with Express handles API routing and proxies all external API requests to protect API keys.

****Environment Variables & API Key Management:**** All sensitive values (API keys, JWT secret, MongoDB URI) must be stored in a `.env` file at the project root. This file must be listed in `.gitignore` and must never be committed to the GitHub repository. A `.env.example`

file with placeholder keys (no real values) must be committed instead, so team members can configure their own environment. Access keys via `process.env.VARIABLE_NAME` using the `dotenv` package.

****Local Development CORS:**** To resolve internal cross-origin issues during development, `vite.config.js` will configure a local proxy to route `/api` requests from the Vite dev server (port 5173) to the Express server (port 5000).

****Production CORS:**** The Vite dev proxy is a development-only tool and is absent in production builds. For production, the Express server must explicitly configure CORS headers using the `cors` npm package, restricting allowed origins to the deployed frontend domain. Alternatively, Express can serve the built React static files directly, eliminating CORS entirely.

****Security:****

- Session security is maintained via JWT stored in HttpOnly cookies with the `SameSite=Lax` attribute. `SameSite=Lax` is the correct setting for a MERN stack where the frontend and backend may be served from different origins or ports; it allows cookie transmission on top-level navigations while still protecting against most CSRF vectors. `SameSite=Strict` must not be used as it will silently break cookie transmission on any cross-site redirect.
- The `Secure` cookie attribute must be set in production (HTTPS) but may be omitted in local HTTP development.
- All incoming request bodies must be validated server-side. Use `express-validator` or manual checks on all route handlers to guard against NoSQL injection (e.g., reject fields containing MongoDB operator keys such as `$where`, `$gt`). Client-side Zod validation is not a substitute for server-side validation.

****HTTP Status Code Conventions**** (all modules must follow):

Scenario	Status Code
Success with response body	200
Resource successfully created	201
Success with no body (e.g., delete)	204
Malformed request / validation failure	400
Not authenticated (no/invalid token)	401
Authenticated but not authorised	403
Resource not found	404
Server-side error	500

2.5 Database

****MongoDB.**** Data structures are explicitly normalised to prevent document bloat and ensure separation of concerns:

- **Goals:** Stored in a separate collection referenced by `userId` to handle unbounded 1-to-Many growth. A user may have many goals, and querying or updating individual goals without rewriting the parent document is the primary driver.
- **CashFlow:** Kept in a separate collection to allow the `expenses` array to be queried and updated independently without rewriting the entire main User document. This also demonstrates normalisation technique for assessment purposes.
- **Risk Profile:** Embedded directly into the User document as it is a 1-to-1 relationship that is always fetched with the user session.

Indexing: A single-field index on `userId` is created in both the `Goals` and `CashFlow` collections for high-performance queries.

Mongoose Schema Definitions:

User collection:

```

...
{
  name:      String, required
  email:     String, required, unique, lowercase
  passwordHash: String, required
  isActive:  Boolean, default: true
  riskProfile: {
    profile: String, enum: ['Conservative','Moderate','Aggressive'], default: null
    allocation: {
      bonds: Number
      equities: Number
      cash: Number
    }
  }
  score: Number
},
createdAt: Date (auto via timestamps)
updatedAt: Date (auto via timestamps)
}
...

```

CashFlow collection:

```

...
{
  userId: ObjectId, ref: 'User', required, indexed
  netIncome: Number, required, default: 0
  expenses: [{
    label: String, required
    amount: Number, required
  }]
  createdAt: Date
  updatedAt: Date
}
...

```

Goals collection:

...

```
{
  userId:      ObjectId, ref: 'User', required, indexed
  title:       String, required
  targetAmount: Number, required
  savedAmount: Number, required, default: 0
  targetDate:  Date, required
  createdAt:   Date
  updatedAt:   Date
}
```

2.6 Version Control

GitHub for collaborative development and code hosting.

Branch naming convention: ``feature/<module-number>-<short-description>`` (e.g., ``feature/1-jwt-auth``, ``feature/3-goal-crud``).

Merge policy: All merges to ``main`` must be made via a Pull Request. Direct commits to ``main`` are strictly prohibited. Each PR requires at least one review approval from another team member before merging. The team leader (Tan Yin Xuan) has final merge authority.

3. UI/UX Design Specifications

The system utilises a custom SCSS-compiled variable palette to ensure a premium, modern SaaS aesthetic.

Variable	🌞 Light Mode ("Executive Clarity")	🌙 Dark Mode ("Midnight Premium")
``$body-bg``	``#F8FAFC`` (Soft Slate White)	``#0F172A`` (Deep Midnight Slate)
``$card-bg``	``#FFFFFF`` (Pure White)	``#1E293B`` (Elevated Slate)
``$primary``	``#2563EB`` (Royal Blue)	``#3B82F6`` (Brighter Neon Blue)
``$success``	``#10B981`` (Vibrant Emerald Green)	``#34D399`` (Luminous Mint Green)
``$danger``	``#EF4444`` (Crisp Red)	``#F87171`` (Soft Coral Red)
``$text-color``	``#0F172A`` (Deep Slate)	``#F8FAFC`` (Off-White)
``$text-muted``	``#64748B`` (Cool Gray)	``#94A3B8`` (Light Slate)

Loading States: Every module must implement skeleton loading UI for all data-dependent views using Bootstrap's `.placeholder`` utility classes. This applies to all TanStack Query ``isLoading`` states. A consistent spinner component must be used for action-triggered loading (e.g., form submissions).

****Accessibility (a11y) Baseline:**** All interactive elements must be keyboard-navigable. Form inputs must have associated ``<label>`` elements. Color alone must not be the sole indicator of state (e.g., error states must include an icon or text, not just a red border). Color contrast ratios must meet WCAG AA standards against the palette above.

4. Module Specifications & Feature Breakdown

Module 1: Authentication & Core Infrastructure

****Assigned to:**** Ajax Kang AJ (server scaffolding, JWT, auth routes) & Lai Onn Wang (DB connection, error handling, middleware)

****Backend Scaffolding:**** Build the foundational Express.js server, establish the Mongoose database connection, set up global error-handling middleware, and configure the JWT middleware (`HttpOnly + SameSite=Lax`).

****Secure Authentication:**** Implement registration and login forms using `react-hook-form` and `Zod`. Passwords must be hashed using `bcrypt` on the server with a minimum cost factor of ****12 salt rounds****.

****JWT Lifecycle:**** JWTs must be issued with an expiry of ****7 days**** (`expiresIn: '7d'`). There is no silent refresh in this implementation; upon token expiry, the user is logged out and redirected to the login page. This behaviour must be handled by the `GET /api/auth/me` endpoint returning a 401, which the Zustand auth store interprets as a session-expired logout event.

****Session Rehydration (Critical):**** To prevent session loss on page refresh (as Zustand is in-memory), implement a `GET /api/auth/me` endpoint. On application initialisation (via a top-level `useEffect`), this endpoint validates the `HttpOnly` cookie and repopulates the Zustand auth store. This endpoint must also verify `isActive: true` before returning user data — if the account is deactivated, it must return `401`.

****Protected Routes:**** Enforce session timeouts client-side via a React Router `<ProtectedRoute/>` wrapper that reads from the Zustand auth store.

****All Protected Backend Routes:**** Every Express route handler that requires authentication must execute two checks in the JWT middleware before processing: (1) token validity, and (2) `user.isActive === true`. A deactivated user with a valid, unexpired JWT must receive a `403 Forbidden` response.

****Profile Control & Cascade Deletion:**** Users can view dynamic profiles, update personal information, and change passwords. On permanent account deletion, the backend must execute a cascade delete using `Promise.all([ Goals.deleteMany({ userId }), CashFlow.deleteOne({ userId }) ])` to remove all associated documents and prevent orphaned data. Temporary account deactivation sets `isActive: false` on the User document; this does not delete any data.

Module 2: Cash Flow & Budget Baseline

****Assigned to:**** Tan Yin Xuan

****Schema Definition:**** Each user has a single CashFlow document containing their `netIncome` and an array of fixed `expenses` entries (see schema in Section 2.5).

****New User Null State Guarantee:**** If no CashFlow document exists for the authenticated user, the backend `GET /api/cashflow` endpoint must return the following defined empty state rather than a 404, to prevent frontend runtime crashes in dependent modules:

```
```json
{ "netIncome": 0, "expenses": [], "disposableIncome": 0 }
```
```

****Income & Expense Tracking:**** Users establish a financial baseline by inputting their monthly net income and recurring fixed expenses.

****Disposable Income Calculation:**** The backend dynamically calculates and appends `disposableIncome` to the API response using the formula:

```
```
Disposable Income = Net Income – Sum(all expense amounts)
```
```

Both Module 3 and Module 6 will consume this calculated value via the shared TanStack Query hook `useCashFlow()`. This hook must never be duplicated; all modules import it from a shared `/src/hooks/useCashFlow.js` file.

Module 3: Goal-Based Planning

****Assigned to:**** Lee Yi Chen

****Interconnected Tracking & Validation:**** Users set financial goals and log saved amounts. The system cross-references each goal's required monthly saving against the user's available disposable income from `useCashFlow()`. Required monthly saving is derived as:

```
```
Required Monthly Saving = (Target Amount – Saved Amount) / Months Until Target Date
```
```

If required monthly saving across all active goals exceeds disposable income, a non-blocking UI warning banner is displayed.

****Data Structure:**** Each Goal document contains a `savedAmount` field (Number) that is overwritten on each update. The Goals collection is bounded only by the number of goals a

user creates; no programmatic limit is enforced, but the UI will display a soft warning after 20 active goals for UX clarity.

****Empty State Prompt:**** If `useCashFlow()` returns the new-user empty state (`disposableIncome: 0`` and no income set), the UI will render a prompt directing the user to complete their financial baseline first. Goal creation will be disabled until a non-zero `netIncome`` is recorded.

****Goal Management:**** Full CRUD operations. Goals are stored in the separate, ``userId``-indexed Goals collection.

Module 4: Investment Strategy

****Assigned to:**** Cheng Tze Xuan

****Risk Profile Assessment:**** A dynamic React questionnaire evaluates risk tolerance and time horizon.

****New User Null State:**** If `user.riskProfile.profile`` is ``null``, the UI must render an onboarding prompt directing the user to complete the questionnaire, rather than attempting to display empty allocation data.

****Questionnaire & Scoring Design:**** The questionnaire consists of ****6 questions****, each with answer options worth 0–5 points, for a maximum total score of 30. The score-to-profile mapping is:

| Score Range | Profile |
|-------------|--------------|
| 0 – 10 | Conservative |
| 11 – 20 | Moderate |
| 21 – 30 | Aggressive |

The raw answers and total score are sent to ``POST /api/investment/profile``. The backend computes the profile and returns the result; it does not trust a client-computed profile string.

****Explicit Output Schema:**** The result is embedded into the User document:

```
``json
{
  "profile": "Moderate",
  "score": 16,
  "allocation": { "bonds": 40, "equities": 50, "cash": 10 }
}
```

Predefined allocation breakdowns per profile:

- ****Conservative:**** bonds: 60, equities: 20, cash: 20
- ****Moderate:**** bonds: 40, equities: 50, cash: 10

- **Aggressive:** bonds: 10, equities: 80, cash: 10

Optimised Storage: This calculated profile object is embedded directly into the main User document (1-to-1 relationship, always needed with the session).

Module 5: Market Insights & Analysis

Assigned to: Tan Yin Xuan

Split API Proxy Strategy: All requests to external financial APIs are routed through the Node.js backend (`/api/market/...`) to prevent CORS errors and protect API keys (stored in `.env`, never in frontend code).

- **CoinGecko** is used for cryptocurrency historical line charts. The specific endpoint used is `GET /coins/{id}/market_chart?vs_currency=usd&days={days}`. This endpoint is available on the free tier.

- **Finnhub** is used for traditional equity quotes (`GET /quote?symbol=...`) and market news (`GET /news?category=general`). The Finnhub free tier enforces a limit of **60 API calls per minute**.

Rate Limit Handling (Finnhub): The Express proxy layer for Finnhub routes must implement a server-side in-memory request counter. If the rate limit is approached (>50 calls/min), the proxy must return a `429 Too Many Requests` response to the frontend. The frontend TanStack Query configuration must handle `429` errors by displaying the styled fallback component (not a crash) and setting a retry delay of 60 seconds.

Expected API Response Shapes (for Phase 1 mocking):

CoinGecko `/coins/{id}/market_chart` response shape:

```
```json
{
 "prices": [[1700000000000, 36000.12], [1700086400000, 36500.45]],
 "market_caps": [[1700000000000, 700000000000]],
 "total_volumes": [[1700000000000, 180000000000]]
}
```
```

Finnhub `/quote` response shape:

```
```json
{
 "c": 182.63,
 "d": 1.25,
 "dp": 0.69,
 "h": 183.10,
 "l": 181.20,
 "o": 181.50,
 "pc": 181.38,

```

```
"t": 1700000000
}
```

Phase 1 mock files in ``/src/mocks/`` must exactly replicate these shapes.

**\*\*Market Trend Visualisation:\*\*** Render responsive line charts using Chart.js based on the proxy data. TanStack Query caches fetched data per timeframe key (e.g., ``['market', 'bitcoin', '7d']``) to prevent redundant calls.

**\*\*API Failure Handling:\*\*** UI must gracefully handle API unavailability using TanStack Query's ``isError`` states, rendering styled fallback card components rather than blank charts.

---

### ### Module 6: Integrated Financial Calculators

**\*\*Assigned to:\*\*** Veronica Chieng Xin Ru

**\*\*Context-Aware ROI Calculator:\*\*** Pulls target amounts directly from active user goals by reading TanStack Query's cached Goals data (query key: ``['goals']``). Disposable income is read via ``useCashFlow()``.

The ROI calculator computes return on a lump-sum investment using the compound interest formula to first derive the Future Value, then calculates ROI as follows:

**\*\*Step 1 — Derive Future Value using compound interest:\*\***

...

$$A = P \times (1 + r/n)^{(n \times t)}$$

...

Where:

- ``A`` = Future Value
- ``P`` = Principal (lump sum invested)
- ``r`` = Annual interest rate as a decimal (e.g., 0.05 for 5%)
- ``n`` = Compounding frequency per year (see table below)
- ``t`` = Time in years

**\*\*Step 2 — Calculate ROI:\*\***

...

$$\text{ROI (\%)} = ((A - P) / P) \times 100$$

...

Note: parentheses are mandatory in code to enforce operator precedence — ``(A - P) / P``, not ``A - P / P``.

**\*\*Compounding Frequency Options (user-selectable in UI):\*\***

```
| Label | n value |
|---|---|
```

Annually	1
Quarterly	4
Monthly	12
Daily	365

**\*\*Investment Comparison Tool:\*\*** A side-by-side comparison displaying potential future values across three investment vehicles with the following preset default annual interest rates (user-overridable):

Vehicle	Default Annual Rate
Fixed Deposit	3.5%
Unit Trust	6.0%
Index Fund	8.0%

All calculations are performed client-side. Results must be rounded to 2 decimal places before display.

---

## ## 5. Assessment Milestones & Deliverables

### ### 5.1 Phase 1: Front-End Prototype (Week 8)

**\*\*Objective:\*\*** Deliver the complete client-side UI/UX with no backend integration.

**\*\*Data Mocking Strategy:\*\*** The React application will rely strictly on hardcoded JSON mock data housed in a dedicated `/src/mocks/` directory. Mock files must exactly replicate the JSON response shapes documented in Section 4 (Module 5) and the API contracts implied by each module's backend spec. Mock data must be consumed through the same custom TanStack hooks (`useCashFlow()`, `useGoals()`, etc.) that will later call the real backend, so that Phase 2 integration requires only changing the data source, not the component code.

**\*\*Submissions:\*\*** React/Vite build via SPECTRUM, Part 1 of Project Report (detailing Vite, Zustand, TanStack Query, Zod, and SCSS customisation), and Peer Evaluation Forms.

### ### 5.2 Phase 2: Full System Integration (Week 14)

**\*\*Objective:\*\*** Deliver the fully functional system (React Frontend + Node/Express Backend + MongoDB + External APIs).

**\*\*Submissions:\*\*** Final Project Report via SPECTRUM, GitHub repository link, and exported MongoDB JSON data file.

**\*\*Report Focus Requirements:\*\*** The report must include in-depth explanations and justifications of:

1. MERN system architecture, module interaction diagram, and Vite proxy CORS configuration.
2. Production CORS strategy (how Express handles cross-origin requests outside of the Vite dev proxy).
3. Database normalisation decisions: embedding Risk Profile, separating the CashFlow document, and cascade deletes.
4. MongoDB indexing strategy and the specific indexes created for `userId`.
5. State management architecture: the Zustand vs. TanStack Query boundary rule, and the `staleTime`/`gcTime` values chosen per query type.
6. Session rehydration strategy (`GET /api/auth/me`) to resolve in-memory Zustand state loss on page refresh, including the `isActive` check.
7. JWT lifecycle: token expiry duration and the logout-on-expiry flow.
8. Security implementation: the split Node.js API proxy (CoinGecko/Finnhub), JWT HttpOnly with SameSite=Lax, bcrypt cost factor 12, NoSQL injection mitigation, and protected React routes.
9. `.env` management strategy and the `.env.example` convention for team collaboration.
10. Evidence of SCSS Bootstrap compilation (`vite.config.js`, main SCSS entry file).